

A Hands-on Exercise: Programming for Distributed Systems (Cluster) & MPI

Exercise Theme

MPI with main focus on **explicit communication between independent address spaces**. This exercise intentionally makes you **break programs, diagnose hangs, and fix them**, because that is how distributed programming becomes real.

Learning Objectives

By the end of this exercise, you will be able to:

1. Explain cluster architecture using Ubuntu VMs in VirtualBox (master/slave, bridged networking, NFS, SSH).
2. Compile and run MPI programs on a single node and across a VM-based cluster.
3. Correctly use `MPI_Init`, `MPI_Finalize`, `MPI_Comm_rank`, `MPI_Comm_size`, `MPI_Get_processor_name`.
4. Understand blocking semantics, message matching (source/tag/communicator), and why output order is arbitrary.
5. Diagnose common deadlock patterns and fix them using ordering, `MPI_Sendrecv`, or non-blocking calls.
6. Safely handle unknown message sizes using `MPI_Status`, `MPI_Get_count`, and `MPI_Probe`.
7. Measure communication/computation time using `MPI_Wtime` and interpret results via a basic cost model.
8. Apply a parallelization strategy: decomposition (block vs cyclic), load balance, and communication design.

Environment Requirements

You must have a working VM-based cluster (minimum 2 nodes: master + slave1), a detailed setup guide is available on GCR as well as in the appendix at the end of this document. Here is a summary of the setup:

- VirtualBox Ubuntu/Kali VMs with **Bridged Adapter** networking
- Static IPs and hostnames (e.g., `master`, `slave1`)
- NFS share mounted at `/mirror` on all nodes
- Password-less SSH between nodes (recommended via shared `/mirror/.ssh`)

- MPICH installed (`mpicc`, `mpiexec` available)

Note

All code in this exercise should be placed inside `/mirror/exercise/` so every node sees the same files.

Part 0: Cluster Architecture Overview (Ubuntu/Kali VMs on VirtualBox)

What is the “cluster” we are using?

This course uses a Beowulf-style cluster built from Ubuntu Virtual Machines (VMs) inside VirtualBox. Each VM behaves like a separate computer with its own OS, RAM, and process space.

Warning

NFS file sharing is not memory sharing.

A file in `/mirror` is shared. Your pointers and variables are not.

Shared Memory vs Distributed Memory (diagram)

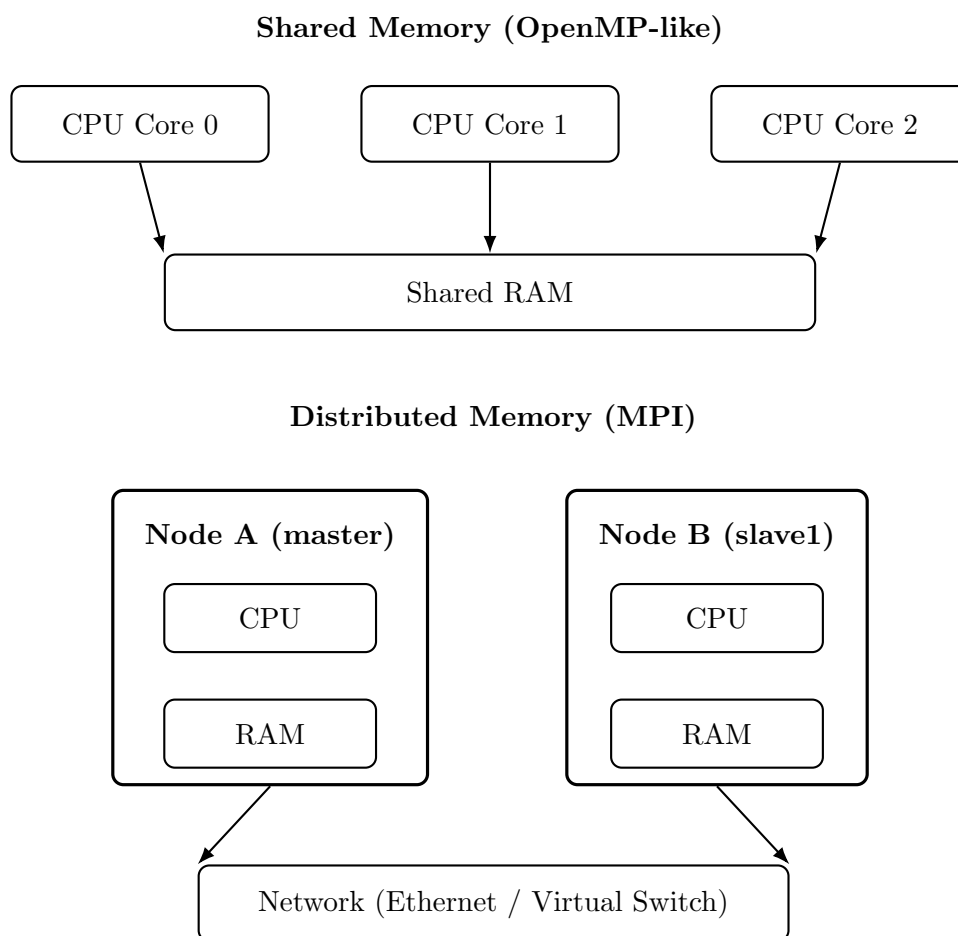


Figure 1: Architecture contrast: shared-memory vs distributed-memory execution.

Logical vs Physical view (diagram)

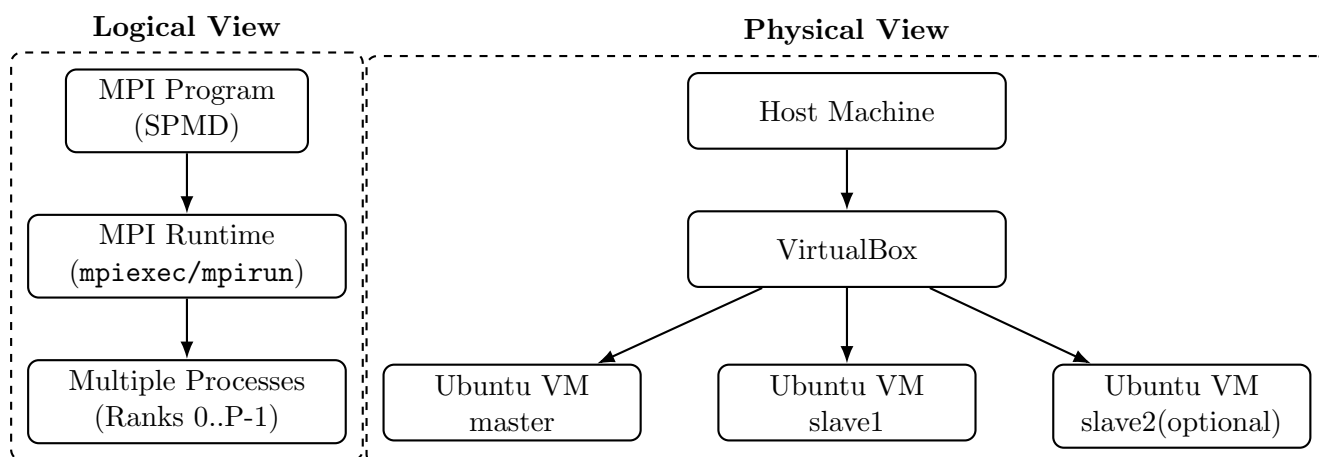


Figure 2: Logical MPI execution vs physical VM deployment.

MPI launch sequence diagram

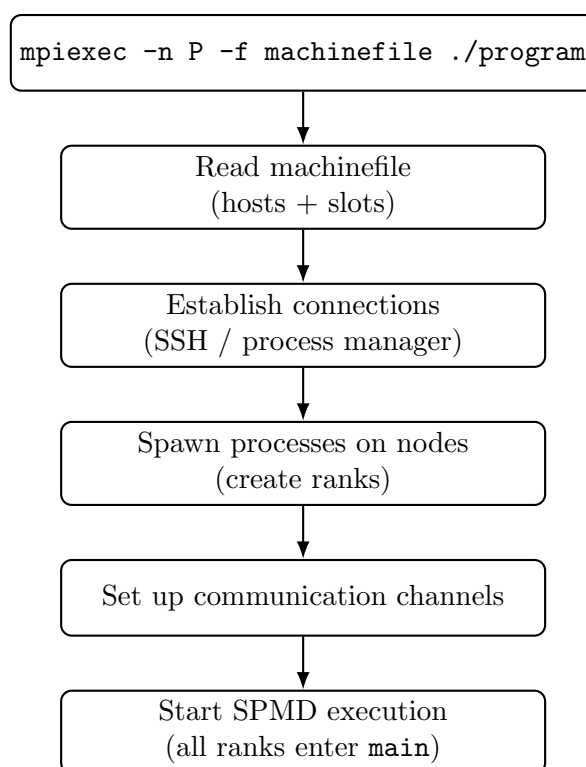


Figure 3: High-level launch flow when running an MPI job on a cluster.

Architecture verification activity

On **each node** (master and slave1), record:

1. hostname: _____

2. ip addr (IP): _____

On **master**:

1. Ping by name: `ping -c 2 slave1`

Result (success/fail + one line note): _____

2. SSH into slave: `ssh slave1`

Result: _____

3. Verify NFS: `ls -ld /mirror` and create a test file `/mirror/exercise/test.txt`

Confirm file visible on slave1: _____

Checkpoint

You should be able to SSH without password and see the same `/mirror` on all nodes. If not, stop here and fix the cluster before coding.

1 Part 1: SPMD Reality Check (Same code, different memory)

Create directory:

```
mkdir -p /mirror/exercise && cd /mirror/exercise
```

1.1 Program 1: Rank/Size/Processor mapping

Create p1_map.c:

```
1 #include <stdio.h>
2 #include <mpi.h>
3
4 int main(int argc, char **argv) {
5     int rank, size;
6     char name[MPI_MAX_PROCESSOR_NAME];
7     int len = 0;
8
9     MPI_Init(&argc, &argv);
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11    MPI_Comm_size(MPI_COMM_WORLD, &size);
12    MPI_Get_processor_name(name, &len);
13
14    printf("[rank %d/%d] running on %s\n", rank, size, name);
15
16    MPI_Finalize();
17    return 0;
18 }
```

Compile and run:

```
mpicc p1_map.c -o p1_map
mpiexec -n 4 ./p1_map
```

Now create a machinefile machinefile:

```
master:2
slave1:2
```

Run across the cluster:

```
mpiexec -n 4 -f machinefile ./p1_map
```

Answer: Is rank 0 always on master? Explain in one line.

1.2 Program 2: “Same variable name” experiment

Create p2_memory.c:

```
1 #include <stdio.h>
2 #include <mpi.h>
3
4 int main(int argc, char **argv) {
5     int rank, size;
6     int x = 5;
7
8     MPI_Init(&argc, &argv);
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
11
12     if (rank == 0) x = 100;
13
14     printf("[rank %d] x=%d (private memory)\n", rank, x);
15
16     MPI_Finalize();
17     return 0;
18 }
```

Question: Why do other ranks still print 5?

2 Part 2: Blocking semantics + message matching (source/-tag/comm)

2.1 Program 3: Basic blocking send/recv

Create p3_p2p_int.c:

```

1 #include <stdio.h>
2 #include <mpi.h>
3
4 int main(int argc, char **argv) {
5     int rank;
6     MPI_Status status;
7
8     MPI_Init(&argc, &argv);
9     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
10
11     if (rank == 0) {
12         int data = 42;
13         MPI_Send(&data, 1, MPI_INT, 1, 7, MPI_COMM_WORLD);
14         printf("[rank 0] sent %d with tag=7\n", data);
15     } else if (rank == 1) {
16         int recv = -1;
17         MPI_Recv(&recv, 1, MPI_INT, 0, 7, MPI_COMM_WORLD, &status);
18         printf("[rank 1] received %d from source=%d tag=%d\n",
19               recv, status.MPI_SOURCE, status.MPI_TAG);
20     }
21
22     MPI_Finalize();
23     return 0;
24 }

```

Run:

```

mpicc p3_p2p_int.c -o p3_p2p_int
mpiexec -n 2 ./p3_p2p_int

```

2.2 Output ordering reality

Run `mpiexec -n 8 ./p1_map` multiple times. Note output order differences.

Question: Why is output order not guaranteed?

2.3 Buffer reuse experiment (small vs large)

Create p4_buffer_reuse.c (small + large messages and overwrite):

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 static void fill(int *a, int n, int v) {
6     for (int i=0; i<n; i++) a[i]=v;
7 }
8
9 static int checksum(const int *a, int n) {
10     long long s=0;
11     for (int i=0; i<n; i++) s += a[i];

```

```
12     return (int)(s % 1000000007);
13 }
14
15 int main(int argc, char **argv) {
16     int rank;
17     MPI_Init(&argc, &argv);
18     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
19
20     const int n = (argc > 1) ? atoi(argv[1]) : 16; // try 16 then
21     1000000
22     int *buf = (int*)malloc(sizeof(int)*n);
23
24     if (rank == 0) {
25         fill(buf, n, 1);
26         MPI_Send(buf, n, MPI_INT, 1, 0, MPI_COMM_WORLD);
27         fill(buf, n, 9); // overwrite immediately after send
28         printf("[rank 0] sent then overwrote buffer, n=%d, checksum(now
29             )=%d\n", n, checksum(buf,n));
30     } else if (rank == 1) {
31         MPI_Recv(buf, n, MPI_INT, 0, 0, MPI_COMM_WORLD,
32             MPI_STATUS_IGNORE);
33         printf("[rank 1] received n=%d checksum=%d\n", n, checksum(buf,
34             n));
35     }
36
37     free(buf);
38     MPI_Finalize();
39     return 0;
40 }
```

Run:

```
mpicc p4_buffer_reuse.c -o p4_buffer_reuse
mpiexec -n 2 ./p4_buffer_reuse 16
mpiexec -n 2 ./p4_buffer_reuse 1000000
```

Discussion prompt (write 2 lines): Why can small messages behave “more forgivingly” than large ones?

3 Part 3: Deadlocks & how to fix

3.1 Deadlock A: Symmetric send-first

Create `p5_deadlock_sendfirst.c`:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 int main(int argc, char **argv) {
6     int rank;
7     MPI_Init(&argc, &argv);
8     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
9
10    const int n = (argc > 1) ? atoi(argv[1]) : 16; // try 16 then
11    1000000
12    int *buf = (int*)malloc(sizeof(int)*n);
13
14    if (rank == 0) {
15        MPI_Send(buf, n, MPI_INT, 1, 0, MPI_COMM_WORLD);
16        MPI_Recv(buf, n, MPI_INT, 1, 0, MPI_COMM_WORLD,
17                MPI_STATUS_IGNORE);
18        printf("[rank 0] completed\n");
19    } else if (rank == 1) {
20        MPI_Send(buf, n, MPI_INT, 0, 0, MPI_COMM_WORLD);
21        MPI_Recv(buf, n, MPI_INT, 0, 0, MPI_COMM_WORLD,
22                MPI_STATUS_IGNORE);
23        printf("[rank 1] completed\n");
24    }
25
26    free(buf);
27    MPI_Finalize();
28    return 0;
29 }
```

Run:

```

mpicc p5_deadlock_sendfirst.c -o p5_deadlock_sendfirst
mpiexec -n 2 ./p5_deadlock_sendfirst 16
mpiexec -n 2 ./p5_deadlock_sendfirst 1000000
```

Task: Record which run hangs and why (2 lines).

3.2 Fix 1: Enforced ordering (recv-first on one side)

Create `p6_fix_order.c` and fix by doing recv-first at rank 1 (or vice versa).

Explain in one line why it works: _____

3.3 Fix 2: MPI_Sendrecv

Create `p7_fix_sendrecv.c` using `MPI_Sendrecv` (single call per rank).

Explain in one line why it is safer: _____

4 Part 4: Unknown message sizes (Status, Get_count, Probe)

4.1 Problem

In real programs, receiver often does not know the message length in advance. Allocating “huge buffers” is wasteful and unsafe.

4.2 Program 4: Probe + dynamic receive

Create p8_probe.c:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 int main(int argc, char **argv) {
6     int rank;
7     MPI_Status status;
8
9     MPI_Init(&argc, &argv);
10    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
11
12    if (rank == 0) {
13        int n = (argc > 1) ? atoi(argv[1]) : 13;
14        int *a = (int*)malloc(sizeof(int)*n);
15        for (int i=0; i<n; i++) a[i]=i+1;
16        MPI_Send(a, n, MPI_INT, 1, 99, MPI_COMM_WORLD);
17        free(a);
18    } else if (rank == 1) {
19        MPI_Probe(0, 99, MPI_COMM_WORLD, &status);
20
21        int count = 0;
22        MPI_Get_count(&status, MPI_INT, &count);
23
24        int *b = (int*)malloc(sizeof(int)*count);
25        MPI_Recv(b, count, MPI_INT, 0, 99, MPI_COMM_WORLD,
26                MPI_STATUS_IGNORE);
27
28        long long sum = 0;
29        for (int i=0; i<count; i++) sum += b[i];
30        printf("[rank 1] received count=%d, sum=%lld\n", count, sum);
31
32        free(b);
33    }
34
35    MPI_Finalize();
36    return 0;
37 }
```

Run:

```
mpicc p8_probe.c -o p8_probe
mpiexec -n 2 ./p8_probe 1000
```

Question: Why is MPI_Probe better than “always receive into a big buffer”?

5 Part 5: Non-blocking communication (overlap is a design choice)

5.1 Program 5: Isend/Irecv with two modes

Create p9_nonblock_overlap.c:

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <mpi.h>
4
5  static void fake_compute(long long iters) {
6      volatile double x = 0;
7      for (long long i=0; i<iters; i++) x += (i % 7) * 0.0000001;
8  }
9
10 int main(int argc, char **argv) {
11     int rank;
12     MPI_Request req;
13     MPI_Status st;
14
15     MPI_Init(&argc, &argv);
16     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
17
18     int n = (argc > 1) ? atoi(argv[1]) : 1000000;
19     int mode = (argc > 2) ? atoi(argv[2]) : 0; // 0=wait immediately,
        1=overlap
20     long long iters = (argc > 3) ? atoll(argv[3]) : 200000000LL;
21
22     int *buf = (int*)malloc(sizeof(int)*n);
23     for (int i=0; i<n; i++) buf[i]=rank;
24
25     double t0 = MPI_Wtime();
26
27     if (rank == 0) {
28         MPI_Isend(buf, n, MPI_INT, 1, 1, MPI_COMM_WORLD, &req);
29         if (mode == 1) fake_compute(iters);
30         MPI_Wait(&req, &st);
31     } else if (rank == 1) {
32         MPI_Irecv(buf, n, MPI_INT, 0, 1, MPI_COMM_WORLD, &req);
33         if (mode == 1) fake_compute(iters);
34         MPI_Wait(&req, &st);
35     }
36
37     double t1 = MPI_Wtime();
38     if (rank == 0) {
39         printf("[rank 0] n=%d mode=%d total_time=%.6f sec\n", n, mode,
40             (t1 - t0));
41     }
42     free(buf);
43     MPI_Finalize();
44     return 0;
45 }

```

Run:

```

mpicc p9_nonblock_overlap.c -o p9_nonblock_overlap
mpiexec -n 2 ./p9_nonblock_overlap 1000000 0

```

```
mpiexec -n 2 ./p9_nonblock_overlap 1000000 1
```

Observation: Does mode=1 always win? Why/why not?

6 Part 6: Parallelization Strategy (Decomposition + Load Balance + Communication Design)

6.1 The strategy framework

For a distributed program, you must decide:

1. **Decomposition:** how to split work/data (block, cyclic, etc.)
2. **Mapping:** which rank gets which piece
3. **Communication:** what to exchange, how often, what sizes
4. **Cost:** communication vs computation (latency/bandwidth)

We will implement the same problem using two decompositions and compare.

6.2 Problem: Distributed sum of N integers

Compute:

$$S = \sum_{i=0}^{N-1} a[i]$$

where rank 0 initially owns the full array.

6.3 Decomposition A: Block partition (contiguous chunks)

Each rank gets a contiguous segment. Good for locality, minimal metadata.

6.4 Decomposition B: Cyclic partition (round-robin)

Each rank gets indices $i = \text{rank}, \text{rank}+P, \text{rank}+2P, \dots$. Often improves load balance when work per element varies.

Note

In this exercise, we'll create **artificial imbalance** to show why cyclic sometimes helps.

6.5 Program 6: Block vs cyclic with artificial imbalance + timing

Create `p10_sum_decomp.c`:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <mpi.h>
4
5 static void heavy_work(int v) {
6     // Artificial imbalance: make some values "expensive"
7     // NOTE: This is intentionally wasteful to create uneven workload.
8     volatile double x = 0;
9     int iters = (v % 97 == 0) ? 5000 : 50;
10    for (int i=0; i<iters; i++) x += (i * 0.0000001);
11 }
12
13 int main(int argc, char **argv) {
14     int rank, size;
15     MPI_Init(&argc, &argv);
16     MPI_Comm_rank(MPI_COMM_WORLD, &rank);

```

```

17 MPI_Comm_size(MPI_COMM_WORLD, &size);
18
19 long long N = (argc > 1) ? atoll(argv[1]) : 5000000LL;
20 int mode = (argc > 2) ? atoi(argv[2]) : 0; // 0=block, 1=cyclic
21
22 int *a = NULL;
23 if (rank == 0) {
24     a = (int*)malloc(sizeof(int)*N);
25     for (long long i=0;i<N;i++) a[i] = (int)(i + 1);
26 }
27
28 double t0 = MPI_Wtime();
29
30 long long local_sum = 0;
31
32 if (mode == 0) {
33     // ---- BLOCK ----
34     long long base = N / size;
35     long long rem = N % size;
36     long long my_n = base + (rank < rem ? 1 : 0);
37     long long my_off = rank * base + (rank < rem ? rank : rem);
38
39     // Master sends chunks (manual p2p, intentionally not using
40     // Scatter yet)
41     int *buf = (int*)malloc(sizeof(int)*my_n);
42
43     if (rank == 0) {
44         // copy own
45         for (long long i=0;i<my_n;i++) buf[i] = a[my_off + i];
46
47         // send others
48         for (int r=1; r<size; r++) {
49             long long rn = base + (r < rem ? 1 : 0);
50             long long roff = r * base + (r < rem ? r : rem);
51             MPI_Send(a + roff, (int)rn, MPI_INT, r, 10,
52                     MPI_COMM_WORLD);
53         }
54     } else {
55         MPI_Recv(buf, (int)my_n, MPI_INT, 0, 10, MPI_COMM_WORLD,
56                 MPI_STATUS_IGNORE);
57     }
58
59     // local compute with artificial imbalance
60     for (long long i=0;i<my_n;i++) {
61         heavy_work(buf[i]);
62         local_sum += buf[i];
63     }
64
65     free(buf);
66
67 } else {
68     // ---- CYCLIC ----
69     // Rank 0 broadcasts full array size only; each rank computes
70     // its indices.
71     // For a fair comparison, we still need data distribution. We'
72     // ll do a simple broadcast
73     // of the entire array (not scalable, but instructional).
74     // Collectives coming later.

```

```

69     if (rank != 0) a = (int*)malloc(sizeof(int)*N);
70     MPI_Bcast(a, (int)N, MPI_INT, 0, MPI_COMM_WORLD);
71
72     for (long long i=rank; i<N; i+=size) {
73         heavy_work(a[i]);
74         local_sum += a[i];
75     }
76 }
77
78 // Reduce partial sums to rank 0
79 long long global_sum = 0;
80 MPI_Reduce(&local_sum, &global_sum, 1, MPI_LONG_LONG, MPI_SUM, 0,
81           MPI_COMM_WORLD);
82
83 double t1 = MPI_Wtime();
84 if (rank == 0) {
85     printf("[mode=%d] N=%lld P=%d global_sum=%lld time=%.6f sec\n",
86           mode, N, size, global_sum, (t1 - t0));
87 }
88
89 free(a);
90 MPI_Finalize();
91 return 0;
92 }

```

Compile:

```
mpicc p10_sum_decomp.c -o p10_sum_decomp
```

Run (single node):

```
mpiexec -n 4 ./p10_sum_decomp 2000000 0
```

```
mpiexec -n 4 ./p10_sum_decomp 2000000 1
```

Run (cluster):

```
mpiexec -n 4 -f machinefile ./p10_sum_decomp 2000000 0
```

```
mpiexec -n 4 -f machinefile ./p10_sum_decomp 2000000 1
```

6.6 Analysis Questions

1. Which mode was faster under artificial imbalance? Why?

2. What is the major scalability flaw in the cyclic implementation above?

3. In block mode, where can master become a bottleneck?

7 Part 7: One Hybrid Example (MPI + OpenMP + SIMD)

7.1 Kernel: distributed matvec $y = Ax$

MPI: distribute rows of A and gather y .

OpenMP: parallelize local rows.

SIMD: AVX2/FMA vector dot product.

Include the `p11_hybrid_matvec.c` program you already created (MPI+OpenMP+AVX2).
Compile/run:

```
mpicc -O3 -fopenmp -mavx2 -mfma -march=native p11_hybrid_matvec.c -o p11_hybrid_matvec
export OMP_NUM_THREADS=2
mpiexec -n 4 -f machinefile ./p11_hybrid_matvec 4096 4096
```

8 Part 8 (Advanced MPI): Collectives, Irregular Collectives, Datatypes, Scaling, Case Studies

8.1 Why “advanced” MPI matters

As soon as you stop writing toy programs, you need:

- **Collectives** for correctness + performance (avoid rank-0 bottleneck).
- **Irregular distribution** when $N \bmod P \neq 0$.
- **Derived datatypes** to safely send structs and non-contiguous layouts.
- **Nonblocking collectives** to overlap (when overlap is real).
- **Benchmarking + scaling** to prove performance honestly.

8.2 Part A: Collectives vs manual communication

Task: implement distributed sum using `MPI_Scatter` + `MPI_Reduce`.
Create `p12_scatter_reduce.c` (`p1_scatter_reduce`).

Run:

```
mpiexec -n 4 -f machinefile ./p12_scatter_reduce 20000000
```

Prompt: Why can Scatter/Reduce outperform a loop of Sends from rank 0?

8.3 Part B: Irregular data (Scatterv/Gatherv)

Task: make it correct for any N with `counts[]` and `displs[]`.
Create `p13_scatterv_gatherv.c`.

Run:

```
mpiexec -n 3 -f machinefile ./p13_scatterv_gatherv 10
```

Prompt: Interpret `counts[]` and `displs[]` in one paragraph.

8.4 Part C: Derived datatypes (struct safety)

Task: define `MPI_PARTICLE` using `MPI_Type_create_struct`.
Create `p14_particle_dtype.c`.

Run:

```
mpiexec -n 2 -f machinefile ./p14_particle_dtype
```

Prompt: Why is sending a C struct as `MPI_BYTE` unsafe in general?

8.5 Part D: Nonblocking collective (Ireduce) + overlap experiment

Task: compare immediate wait vs overlap compute.

Create `p15_ireduce_overlap.c`.

Run:

```
mpiexec -n 4 -f machinefile ./p15_ireduce_overlap 20000000 0
```

```
mpiexec -n 4 -f machinefile ./p15_ireduce_overlap 20000000 1
```

Prompt: When does overlap help? When does it not?

8.6 Part E: Communication cost (ping-pong) + plotting

Task: measure RTT vs message size and plot bandwidth curve.

Create `p16_pingpong.c` + `plot_pingpong.py`.

Run:

```
mpiexec -n 2 -f machinefile ./p16_pingpong 1 1048576 30 results_pingpong.csv
```

```
python3 plot_pingpong.py results_pingpong.csv pingpong.png
```

Deliver: CSV + two plots.

8.7 Part F: Strong vs weak scaling harness

Task: run strong and weak scaling with `MPI_Allreduce`.

Create `p17_scaling.c`.

Run:

```
mpiexec -n 1 -f machinefile ./p17_scaling strong 20000000 3 results_scaling.csv
```

```
mpiexec -n 2 -f machinefile ./p17_scaling strong 20000000 3 results_scaling.csv
```

```
mpiexec -n 4 -f machinefile ./p17_scaling strong 20000000 3 results_scaling.csv
```

```
mpiexec -n 1 -f machinefile ./p17_scaling weak 5000000 3 results_scaling.csv
```

```
mpiexec -n 2 -f machinefile ./p17_scaling weak 5000000 3 results_scaling.csv
```

```
mpiexec -n 4 -f machinefile ./p17_scaling weak 5000000 3 results_scaling.csv
```

Prompt: Why does strong scaling saturate earlier than weak scaling?

8.8 Case Study 1: Distributed K-means (IoT telemetry)

Goal: data-parallel K-means using `MPI_Allreduce` each iteration.

Create `case_kmeans_iot.c`.

Run:

```
mpiexec -n 4 -f machinefile ./case_kmeans_iot 200000 4 6 15 123
```

Prompt: Identify the per-iteration communication volume and bottleneck.

8.9 Case Study 2: Distributed GEMM (HPC kernel)

Goal: row-block distributed GEMM: `Scatterv(A) + Bcast(B) + local GEMM + Gatherv(C)`.
Create `case_dgemm_blocked.c`.

Run:

```
mpiexec -n 4 -f machinefile ./case_dgemm_blocked 1024 1024 1024 1
```

Prompt: Which communication dominates for large matrices: Scatter, Bcast, or Gather?
Why?

Complete cluster Setup (Used in this Exercise) (VirtualBox + Ubuntu 22.04)

Note

If you already have a cluster setup, you can skip this.

8.10 Goal

A small Beowulf-style cluster is used for this exercise (you can replicate this for exam preparation as well), using two Ubuntu 22.04 Virtual Machines (VMs) in VirtualBox: **master** and **slave1**. By the end, you must be able to run an MPI program across both nodes.

8.11 Prerequisites

Ubuntu 22.04 VM image was downloaded from <https://www.linuxvmimages.com/images/ubuntu-2204>.

- Username: **ubuntu**
- Password: **ubuntu**

8.12 End-State Checklist (You must achieve all)

- VirtualBox networking: **Bridged Adapter** for both VMs
- Static IPs for **master** and **slave1**
- **ping slave1** works from **master** and vice-versa (by name)
- NFS shared folder: **/mirror** exported from **master** and mounted on **slave1**
- Password-less SSH between nodes (for **mpiuser**)
- MPICH installed on both nodes
- MPI hello-world runs across both nodes using a machinefile

8.13 Step 1: Configure VirtualBox Networking (Bridged)

On your Host machine (VirtualBox):

1. Power off the VM.
2. Right click the VM → **Settings** → **Network**.
3. Enable Adapter 1.
4. **Attached to:** select **Bridged Adapter**.
5. Select your real network interface (Wi-Fi or Ethernet).

Boot the VM and confirm internet:

```
ip addr
ip route
ping -c 2 8.8.8.8
```

8.14 Step 2: Set Hostname for Master

On the first VM (you will make it `master`):

```
sudo hostnamectl set-hostname master
```

Log out/in (or reboot) so the hostname updates.

8.15 Step 3: Determine Your Current DHCP Network

On `master`, check DHCP IP and gateway:

```
ip addr
ip route
```

Example (yours may differ):

```
enp0s3: 172.20.161.14/23
default via 172.20.160.1 dev enp0s3
```

This implies subnet: `172.20.160.0/23`. Your static IPs must remain in the same subnet.

8.16 Step 4: Set Static IP on Master (Ubuntu 22.04 Netplan)

Find your interface name:

```
ip link
```

Edit netplan config:

```
ls /etc/netplan
sudo nano /etc/netplan/00-installer-config.yaml
```

Replace DHCP with a static IP (update interface name and IPs to match your LAN):

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 172.20.161.100/23
      gateway4: 172.20.160.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Apply safely:

```
sudo netplan try
sudo netplan apply
```

Verify:

```
ip addr show enp0s3
ip route
ping -c 2 8.8.8.8
```

8.17 Step 5: Clone Master to Create Slave1

On Host machine (VirtualBox):

1. Power off master.
 2. Right click master → Clone.
 3. Choose **Full clone**.
 4. Choose **Generate new MAC addresses**.
 5. Name it: slave1.
- Boot slave1.

8.18 Step 6: Set Hostname + Static IP on Slave1

On slave1:

```
sudo hostnamectl set-hostname slave1
```

Set static IP using netplan (example):

```
sudo nano /etc/netplan/00-installer-config.yaml
```

```
network:
  version: 2
  renderer: networkd
  ethernets:
    enp0s3:
      dhcp4: no
      addresses:
        - 172.20.161.101/23
      gateway4: 172.20.160.1
      nameservers:
        addresses: [8.8.8.8, 1.1.1.1]
```

Apply:

```
sudo netplan try
sudo netplan apply
```

Verify:

```
ip addr
ping -c 2 8.8.8.8
```

8.19 Step 7: Add Name Resolution via /etc/hosts (Both Nodes)

On master:

```
sudo nano /etc/hosts
```

Add (use your actual static IPs):

```
172.20.161.100 master
172.20.161.101 slave1
```

Repeat the same on slave1.

Test:

```
ping -c 3 slave1    # from master
ping -c 3 master    # from slave1
```

8.20 Step 8: Setup NFS Shared Folder (/mirror)

Create folder on both nodes:

```
sudo mkdir -p /mirror
```

Install packages:

on master:

```
sudo apt update
```

```
sudo apt install -y nfs-kernel-server
```

on slave1:

```
sudo apt update
```

```
sudo apt install -y nfs-common
```

Export /mirror on master:

```
sudo nano /etc/exports
```

Add:

```
/mirror 172.20.160.0/23(rw,sync,no_subtree_check)
```

Apply:

```
sudo exportfs -ra
```

```
sudo systemctl restart nfs-kernel-server
```

```
sudo exportfs -v
```

Mount on slave1:

```
sudo mount master:/mirror /mirror
```

```
df -h | grep mirror
```

Test file visibility:

on master:

```
sudo touch /mirror/hello_from_master.txt
```

```
ls -l /mirror
```

on slave1:

```
ls -l /mirror
```

Auto-mount on boot (slave1):

```
sudo nano /etc/fstab
```

Add:

```
master:/mirror /mirror nfs defaults,_netdev 0 0
```

Test:

```
sudo umount /mirror
```

```
sudo mount -a
```

8.21 Step 9: Install SSH Server (Both Nodes)

```
sudo apt install -y openssh-server
```

```
sudo systemctl enable --now ssh
```

8.22 Step 10: Create MPI User (Both Nodes)

Create the same user on both nodes, with home directory in `/mirror`:

```
sudo adduser --home /mirror mpiuser
```

Use the same password on both nodes.

Ownership (best done on master):

```
sudo chown -R mpiuser:mpiuser /mirror
```

8.23 Step 11: Password-less SSH using Shared `/mirror/.ssh`

On master:

```
su - mpiuser
pwd    # should print /mirror
ssh-keygen -t rsa -b 4096
# press Enter for default location: /mirror/.ssh/id_rsa
# press Enter for empty passphrase (convenience)
```

Authorize:

```
cat /mirror/.ssh/id_rsa.pub >> /mirror/.ssh/authorized_keys
chmod 700 /mirror/.ssh
chmod 600 /mirror/.ssh/authorized_keys
```

First-time trust:

```
ssh slave1
exit
```

8.24 Step 12: Install MPICH (Both Nodes)

```
sudo apt install -y build-essential mpich
which mpicc
which mpiexec
mpiexec --version
```

8.25 Step 13: MPI Test Across Cluster

On master (as `mpiuser`), create:

```
nano /mirror/machinefile
```

Contents:

```
master:2
slave1:2
```

Create MPI hello:

```
nano /mirror/mpi_hello.c
```

```

#include <stdio.h>
#include <mpi.h>

int main(int argc, char **argv) {
    int rank, size;
    char name[MPI_MAX_PROCESSOR_NAME];
    int len;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Get_processor_name(name, &len);

    printf("Hello from rank %d/%d on %s\n", rank, size, name);

    MPI_Finalize();
    return 0;
}

```

Compile and run:

```

mpicc /mirror/mpi_hello.c -o /mirror/mpi_hello
mpiexec -n 4 -f /mirror/machinefile /mirror/mpi_hello

```

Expected Output

You should see ranks running on both master and slave1.

8.26 Copy a folder from Documents to /mirror

To copy a folder (recursive copy):

```
cp -r ~/Documents/YourFolder /mirror/
```

If permission denied:

```

sudo cp -r ~/Documents/YourFolder /mirror/
sudo chown -R mpiuser:mpiuser /mirror/YourFolder

```

Or switch user:

```

su - mpiuser
cp -r /home/ubuntu/Documents/YourFolder /mirror/

```

8.27 Troubleshooting Quick Fixes

NFS: “RPC: Program not registered”

on both nodes:

```

sudo apt install -y rpcbind
sudo systemctl enable --now rpcbind

```

on master:

```
sudo systemctl restart nfs-kernel-server
```

SSH keeps asking password

```
chmod 700 /mirror/.ssh  
chmod 600 /mirror/.ssh/authorized_keys
```

Name resolution fails

```
getent hosts master  
getent hosts slave1
```

Checkpoint

Once all steps work, your cluster is ready for MPI Exercise.